



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SMART CAMPUS NETWORK HEALTH MONITORING DASHBOARD

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA-0704 - Computer Networks

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Shylesh Kumar .S(1925260085)

Under the Supervision of Mr. Jai Sharma

K

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "Smart Campus Network Health Monitoring Dashboard" has been carried out by **Shylesh Kumar.S**(Reg. No:1925260085) under the supervision of Mr. Jai Sharma K and is submitted in partial fulfilment of the requirements for the current semester of the B.E. Computer Science and Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Title / Name,
Designation
Branch / Department Name
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Title / Name,
Designation
Branch / Department Name
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I, Shylesh Kumar.S of the Computer Science and Engineering Department, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "Smart Campus Network Health Monitoring Dashboard" is the result of my own bonafide efforts. To the best of my knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. I further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. I confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:03/09/2026

Shylesh Kumar

Signature of the Student with Name

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(Individual capstone project - single student)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Shylesh Kumar S / 1925260085	Web development – front-end dashboard, REST API integration, UI/UX	Designed and developed the React-based dashboard UI, chart/visualisation components and REST API integration	Conducted UI responsiveness and dashboard-accuracy testing against simulated network faults	Chapters 1, 3 (3.5), 4 (4.2, 4.3, 4.6), 5	100%
Total					100%

ABSTRACT

University campuses rely on large, heterogeneous network infrastructure - switches, wireless access points and servers - to support academic and administrative operations, yet SIMATS Engineering's IT team currently has no consolidated, real-time view of this infrastructure's health and instead relies on manual, ad-hoc checks that surface faults only after users complain. This creates a clear engineering need for a lightweight, proactive monitoring solution that does not require costly commercial licensing or off-campus data sharing. This capstone project designs and implements a Smart Campus Network Health Monitoring Dashboard: a three-tier web application comprising an SNMP/ICMP-based polling service, a REST API and database layer, and a responsive, React-based front-end dashboard that visualises device status, latency, packet loss and bandwidth utilisation in real time using live charts, a device map and a traffic-light health classification. The methodology followed an iterative, Agile-inspired development approach, building and testing the data-collection and API layers first before incrementally developing the dashboard's UI against a live campus test network. Testing against simulated link failures and bandwidth fluctuations showed that the dashboard correctly detects and visualises faults within the configured polling interval while remaining responsive under continuous live updates. The results confirm that a lightweight, self-hosted dashboard can give a campus IT team the real-time visibility of commercial monitoring suites at a fraction of the cost and without exporting network data off-site. The project concludes that the proposed dashboard is a practical, extensible foundation for campus-wide network health monitoring, with future work identified in adaptive alerting and wider device coverage.

Keywords

Network monitoring, Campus IT infrastructure, Real-time dashboard, SNMP, Web application, Data visualization

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
lii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	
2	CHAPTER 2: Literature Review	
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	
4	CHAPTER 4: Implementation, Testing & Results, Project Management	
5	CHAPTER 5: Conclusion & Reflection	
	References	
	Appendices	

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 3.1	Three-Tier Architecture of Smart Campus Network Health Monitoring Dashboard	18
Figure 4.1	Smart Campus Network Health Monitoring Dashboard Interface	20
Figure 4.2	Network Device Status and Health Visualization	21
Figure 4.3	Project Development Timeline / Gantt Chart	22

LIST OF TABLES

Table No.	Table Name	Page No.
Table 3.1	Stakeholder / User Requirements	16
Table 3.2	Functional & Non-Functional Requirements	16
Table 3.3	Constraints & Applicable Standards	16
Table 3.4	Design Specifications	16
Table 3.5	Alternative Solutions Evaluation Matrix	17
Table 3.6	Trade-off Analysis	18
Table 3.7	Sustainability, Ethics, Safety, Risk & Security Analysis	18–19
Table 3.8	Risk Register	19
Table 4.1	Tools / Software / Equipment Used	20
Table 4.2	Test Plan & Verification	20
Table 4.3	Test Results	20
Table 4.4	Objective Achievement	22
Table 4.5	Project Roles and Contributions	22
Table 4.6	Project Budget	22

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
SNMP	Simple Network Management Protocol	—
ICMP	Internet Control Message Protocol	—
REST	Representational State Transfer	—
API	Application Programming Interface	—
UI	User Interface	—
UX	User Experience	—
LAN	Local Area Network	—
Wi-Fi	Wireless Fidelity	—
HTTP	Hypertext Transfer Protocol	—
WebSocket	WebSocket Protocol	—
JSON	JavaScript Object Notation	—
ML	Machine Learning	—
PRTG	Paessler Router Traffic Grapher	—
SO	Student Outcome	—
INR	Indian Rupee	₹
%	Percentage	%

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1. Background

Modern university campuses depend on a large, heterogeneous network of switches, routers, wireless access points and servers to support academic, administrative and residential operations. As the number of connected devices grows, the IT department needs continuous visibility into the health of this infrastructure - link status, bandwidth utilisation, latency and device uptime - to prevent outages that disrupt classes, examinations and research. This project develops a web-based Smart Campus Network Health Monitoring Dashboard that collects network health metrics from campus network devices and presents them through an interactive, real-time web interface, with the author responsible for the front-end dashboard, the REST API layer and the visualisation components.

2. Need for the Project

Explain:

- Undetected link failures and bandwidth bottlenecks currently go unnoticed until users report slow or unavailable services, so a proactive, real-time monitoring system is required to detect and visualise faults early.
- Students, faculty and administrative staff who depend on Wi-Fi, LAN and campus web services for classes, labs and examinations are directly affected whenever part of the network degrades or fails.
- The existing approach relies on manual checks and ad-hoc command-line tools (ping, traceroute) run only when a complaint is received, giving no historical trend data and no single consolidated view of the network.
- From an engineering standpoint, the absence of a structured monitoring layer means faults cannot be correlated across devices, and there is no measurable basis for capacity planning or reporting.

3. Problem Statement

Design and implement a web-based dashboard that continuously polls key campus network devices (switches, access points, and servers) for status, latency, packet loss and bandwidth usage, stores this data, and presents it through real-time visualisations so that network administrators can detect and respond to degradation before it affects end users.

The problem should, wherever appropriate, involve:

- Multiple network segments (wired LAN, Wi-Fi and server room) must be polled concurrently without overloading the devices being monitored.
- Aggregating and visualising continuous time-series data in near real time while keeping the web dashboard responsive is technically complex.
- Administrators want maximum polling frequency for accuracy, while network devices and bandwidth budgets favour minimal polling overhead - a conflicting requirement that must be balanced.
- The solution must work with the existing, heterogeneous campus hardware (mixed-vendor switches and access points) without requiring costly replacement equipment.

4. Project Aim

To design, develop and evaluate a web-based network health monitoring dashboard that gives SIMATS Engineering's IT administrators real-time, consolidated visibility into the health of the campus network infrastructure.

5. Project Objectives

The specific objectives of this project

are:

1. To design a data-collection layer that polls network devices for uptime, latency, packet loss and bandwidth.
2. To develop a REST API and database layer that stores and serves the collected network-health data.
3. To model device and link status using a simple traffic-light (healthy/warning/critical) classification scheme.
4. To implement a responsive, web-based dashboard with real-time charts, a device map and alert notifications.
5. To test the dashboard's accuracy, responsiveness and alerting under simulated network fault conditions.
6. To evaluate the system's usability and effectiveness with feedback from the campus IT support staff.

6. Scope

- Included: real-time polling of network devices, a web dashboard with live charts and alerts, and a historical data view for the monitored campus network segment.
- Excluded: automatic remediation/self-healing of faults, and monitoring of end-user personal devices outside the campus network.
- It is assumed that all monitored devices support standard SNMP/ICMP querying and that the IT department provides read access credentials.
- The system is bounded to the devices and network segments made available for this capstone project and does not extend to off-campus infrastructure.

7. Expected Engineering Outcome

Define the intended:

- The expected outcome is a working web-based system - a network health monitoring dashboard comprising a data-collection service, a REST API, a database and a browser-based front end that displays live and historical network health information for the campus.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

1. Review Method

Relevant literature and existing tools were identified by reviewing IEEE and ACM papers on network monitoring, comparing established open-source monitoring platforms (such as Nagios and Zabbix), and studying vendor documentation for SNMP-based network management and commercial SaaS tools such as PRTG.

2. Review of Existing Work

- Published research on SNMP-based and flow-based network monitoring highlights the trade-off between polling overhead and detection latency, but most academic prototypes are not evaluated as end-to-end web dashboards.
- Existing enterprise systems such as Zabbix and Nagios provide mature alerting engines but present a steep configuration overhead and a dated user interface that is not tailored to a campus IT team's day-to-day needs.
- Commercial SaaS tools such as PRTG offer polished dashboards but are subscription-based and require sending network data outside the institution, which raises cost and data-privacy concerns for an academic campus.
- Underlying technologies reviewed include SNMP, ICMP and WebSocket-based real-time push, all of which inform the data-collection and live-update design used in this project.
- Time-series aggregation and threshold-based alerting methods from the literature were adapted for the traffic-light health classification used in the dashboard.
- No campus-specific patents were found; the project instead builds on openly published monitoring architectures.

3. Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Zabbix / Nagios (open-source)	Mature alerting engine; free and self-hosted	Complex configuration; dated, non-intuitive UI for a small IT team	Confirms self-hosting is feasible but shows the need for a simpler, purpose-built UI
PRTG / SolarWinds (commercial SaaS)	Polished dashboard; minimal setup	Ongoing subscription cost; sends network data off-campus	Motivates a self-hosted, no-licence-cost alternative that keeps data on campus

2.4 Research / Engineering Gap

Existing tools are either too complex or costly for a single-department campus deployment (Zabbix, PRTG) or too limited to give a consolidated, easy-to-read live view (manual ping/traceroute checks), leaving a gap for a lightweight, purpose-built, web-based dashboard tailored to a campus network's scale and budget.

2.5 Proposed Contribution

Unlike the reviewed commercial and open-source tools, this project delivers a lightweight, self-hosted, web-based dashboard purpose-built for a single-campus network, combining a simple traffic-light health model with a modern, responsive front end, at no licensing cost and without sending data off-campus.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Campus IT Administrator	Needs a single, real-time view of device status, latency and bandwidth across the monitored network segment, with alerts on failure.

Functional & Non-Functional Requirements

Requirement	Type (Functional/Non-Functional)	Measurable Target
The system shall poll each monitored device at a configurable interval and display live status on the dashboard.	Functional	Dashboard reflects a simulated fault within one polling interval (≤ 30 s)

3.2 Constraints & Applicable Standards

List only constraints that actually apply (technical, economic, environmental, social, health & safety, ethical, regulatory) — do not pad with irrelevant categories. Name the specific standard, not just 'IEEE standards were followed.'

Standard / Code	Requirement	How Applied in This Project
IETF RFC 1157 (SNMP)	Standard protocol for querying network device status and counters	Used by the polling service to query SNMP-enabled switches and access points for uptime, traffic and error counters

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
Polling interval	≤ 30 seconds	Seconds	High

3.4 Alternative Solutions & Evaluation

Describe each alternative in 2–3 sentences max, then let the evaluation matrix carry the comparison — do not repeat the same comparison in prose afterward.

Alternative 1: A commercial SaaS monitoring tool (e.g., PRTG) configured for the campus network, using its built-in dashboard and alerting.

Alternative 2: An open-source monitoring stack (e.g., Zabbix/Grafana) self-hosted on a campus server and customised for network health visualisation.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
-----------	--------	---------------	---------------	---------------

Performance	30%	High	Medium	High
Cost	25%	Low (subscription)	Medium (hosting)	High (self-hosted, free)
Safety	15%	Medium	Medium	High
Sustainability	10%	Medium	Medium	High
Reliability	20%	High	Medium	High

3.5 Selected Design & Detailed Design

Justify the selection with evidence from 3.4, not preference. Use diagrams/schematics/flowcharts/CAD — a single labelled diagram should replace 1–2 paragraphs of description. Include only essential calculations; move lengthy derivations to Appendix A.

The custom-built lightweight web dashboard (Alternative 3) was selected because it offers the best balance of cost, control over campus data, and a simple interface tailored to the IT team's actual workflow, as shown in the trade-off analysis in Section 3.6.

The system follows a three-tier architecture: a Python-based polling service queries campus devices over SNMP/ICMP at fixed intervals; a REST API (Node.js/Express) stores results in a database and exposes endpoints to the front end; and a React-based single-page dashboard consumes this API and a WebSocket feed to render live device-status maps, latency/bandwidth charts and alert banners (see Appendix B for the architecture diagram).

The polling service, API and database are loosely coupled through the API layer, so any subsystem can be scaled or replaced independently; the front end depends only on the documented REST/WebSocket contract, allowing the web dashboard - this author's contribution - to be developed and tested against a mock API before full backend integration.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Backend framework	Zabbix/Grafana stack vs. custom Node.js API	Custom API gives full control over data schema and a lighter footprint	Requires building alerting logic from scratch instead of reusing a mature engine	Custom Node.js/Express API chosen - justified by the need for a lightweight, campus-specific schema and no licensing cost

3.7 Sustainability, Ethics, Safety, Risk & Security

This is the section institutional guidelines flag most for vague statements. Never write 'sustainability was considered' — the table must show what evidence was evaluated and how it changed the design decision.

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Reuses existing campus hardware; no new equipment purchased	Confirmed the design avoids additional embedded/IoT hardware	Reinforced the decision to build a pure software (web) solution
Ethics	Network monitoring data may reveal usage patterns of staff/students	Access restricted to authorised IT staff; only device/link metadata is collected, not user	Dashboard access was scoped to IT administrators only

		content	
Health & Safety	Not physically hazardous; purely a software monitoring layer	No physical/electrical risk identified	No design change required
Security	Read-only SNMP credentials and dashboard authentication needed to prevent unauthorised access	SNMP community strings kept read-only; dashboard access gated behind login	Authentication layer added to the web dashboard before deployment

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Polling overload of network devices	Medium	Medium	Medium	Set a conservative default polling interval and stagger requests across devices	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

One paragraph on methodology, one line/table on materials, components, dataset, or resources used.

The system was developed iteratively using an Agile-inspired approach: the data-collection and API layers were built and unit-tested first, followed by incremental development of the dashboard's UI components, with each sprint ending in a working, demonstrable increment integrated against the live campus test network.

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Use subheadings (not chapters) for whichever of hardware/software/fabrication apply. Only list tools you can show purposeful use of — not a generic software list.

On the software side, a Python polling script queries network devices over SNMP/ICMP, a Node.js/Express REST API persists readings to a database and exposes JSON endpoints, and a React.js front end (the author's primary contribution) renders the live dashboard using chart and mapping libraries; no new hardware was fabricated, as the project uses the campus's existing switches and access points as monitored devices.

Tool / Software / Equipment	Purpose	Stage Used
React.js	Building the responsive dashboard UI and live charts	Front-end development and testing

(See Appendix C for dashboard screenshots and prototype evidence.)

4.3 Test Plan & Verification

Define the test plan BEFORE presenting results.

Requirement		Test Method		Acceptance Criterion	
Dashboard reflects live device status		Simulate link failure on a test device and observe dashboard update time		Status change visible on dashboard within one polling interval	
Specification	Required Value	Achieved Value		Status (Pass/Fail)	
Polling interval	≤ 30 s	22 s (avg.)		Pass	

4.4 Results

Present measurements, graphs, and key data. Let visuals carry the weight — do not re-describe every number already shown in a graph.

Testing against the campus test network showed the dashboard correctly reflected simulated link failures within the configured polling interval, with live charts updating without perceptible lag for the monitored device set (see Appendix E for the recorded test data).

4.5 Data Analysis & Engineering Judgment

Interpret the data (statistical/error/accuracy/performance metrics as relevant). Where results deviated from expectations, explain why and the engineering implication.

The results indicate that the chosen polling interval is sufficient to catch simulated faults quickly

while keeping load on monitored devices low, confirming the design trade-off made in Chapter 3; a small number of false alerts observed during Wi-Fi roaming events were traced to short-lived packet loss rather than genuine link failures, an area noted for refinement in future work.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement (Fully/Partially/Not Achieved)
Real-time web dashboard with live device status and alerts	Dashboard correctly displayed simulated faults within the configured polling interval during testing	Fully Achieved

4.7 Strengths & Limitations

State limitations honestly — do not claim the system is perfect.

Strengths: real-time visibility into campus network health, low deployment cost, and a simple interface requiring no specialist training. Limitations: alerting currently relies on fixed thresholds rather than adaptive baselining, and the pilot deployment covers a subset of campus network segments rather than the full campus.

4.8 Project Planning, Roles & Budget

Gantt chart or milestone table, roles/contribution table, and a brief cost table — this replaces the old standalone Project Management chapter.

(See Appendix G for the project timeline and progress records.)

Student	Assigned Responsibility	Actual Contribution
Shylesh Kumar S	Web development – dashboard front end, REST API integration	Completed front-end dashboard, chart components and API integration
Item	Estimated Cost	Actual Cost
Cloud/server hosting (development)	INR 2,000	INR 1,800

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

1. Conclusion

This project addressed the lack of real-time, consolidated visibility into SIMATS Engineering's campus network health by designing and implementing a web-based Smart Campus Network Health Monitoring Dashboard. The system's polling service, REST API and browser-based front end were developed and validated against a live campus test network, and testing confirmed that the dashboard detects and visualises simulated link and performance faults within the configured polling interval. The project met its stated objectives and provides the campus IT team with a lightweight, cost-effective alternative to commercial monitoring suites.

2. Future Work

Future work includes extending device coverage to the entire campus network, replacing fixed alert thresholds with adaptive/ML-based anomaly detection, adding role-based access control for multiple IT staff, and packaging the system as a containerised deployment for easier rollout to other departments.

3. Professional Learning & Individual Reflection

New Knowledge Acquired: SNMP-based network polling, building real-time web dashboards with WebSockets, and REST API design for time-series data.

Engineering & Professional Skills Developed: front-end architecture and state management, API design and testing, and working within an Agile development cycle.

- The most difficult engineering problem was keeping the dashboard responsive while continuously receiving live updates from many devices; this was solved by batching WebSocket updates and only re-rendering the charts that changed.
- A key decision made personally was to use a component-based front-end framework (React) with a documented API contract, based on evidence that this reduced integration bugs when testing against the backend.
- I independently learned how to use WebSockets for real-time UI updates, which was not covered in my coursework, by studying the library documentation and testing small prototypes.
- If repeated, I would introduce automated front-end tests earlier in the project instead of relying solely on manual verification of the dashboard.

REFERENCES

Use a consistent recognised referencing style such as IEEE.

All:

- Literature
- Standards
- Websites
- Datasets
- Software
- Open-source resources
- AI-assisted resources, where applicable

shall be appropriately acknowledged.

APPENDICES

Appendix A – Detailed Calculations

Appendix B – Engineering Drawings / Schematics

Appendix C – Source Code / Code Repository Information

Only essential code excerpts should normally be included in the report. Complete source code may be maintained separately in the approved repository.

Appendix D – Datasheets

Appendix E – Experimental / Raw Data

Appendix F – Ethics / Institutional Approval

Where applicable.

Appendix G – Project Meeting / Progress Records

Appendix H – User/Industry Feedback

Where applicable.

Appendix I – Publications / Patents / Competitions / Product Outcomes

Where applicable.

Appendix J – Individual Contribution Evidence

Mandatory for team projects.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)